Agents

Create Your Own Agent

Copy page

Build a custom agent for ARC-AGI-3 games

Create AI agents that can play ARC-AGI-3 games by implementing the required interface methods. The following is based off the ARC-AGI-3 Agents repo.

Step 1: Create Your Agent File

First, head over to the ARC-AGI-3-Agents repo and clone it

```
git clone https://github.com/arcprize/ARC-AGI-3-Agents.git
```

Make sure you have your `ARC_API_KEY` populated in your environment variables. You can obtain this key by signing up for an account on the ARC-AGI-3 website.

Next, create a new Python file for your agent inside the `agents/` directory. For this example, let's copy the `random_agent.py` template.

```
cp agents/templates/random_agent.py agents/my_awesome_agent.py
```

Now, modify `agents/my_awesome_agent.py` and rename the class to `MyAwesomeAgent` .

```
import random

# Rename the class
class MyAwesomeAgent(Agent):
    """A simple agent that chooses random actions."""

    def is_done(self, frames: list[FrameData], latest_frame: FrameData) -> boo
        # Your logic to determine if the game is finished
        return latest_frame.state is GameState.WIN

    def choose_action(self, frames: list[FrameData], latest_frame: FrameData)
        # Your custom decision-making logic goes here
        if latest_frame.state in [GameState.NOT_PLAYED, GameState.GAME_OVER]:
            # Start or restart the game
            action = GameAction.RESET
        else:
            # Choose a random action (except RESET)
            action = random.choice([a for a in GameAction if a is not GameActi

        # Add reasoning for simple actions
        if action.is_simple():
            action.reasoning = f"Chose {action.value} randomly"
        # For complex actions, set coordinates
        elif action.is_complex():
            action.set_data({
                "x": random.randint(0, 63),
                "y": random.randint(0, 63),
            })
            action.reasoning = {"action": action.value, "reason": "Random choi

        return action
```

Step 2: Register Your Agent

To make your agent available to run, add an import statement to `agents/__init__.py` and add it to the `AVAILABLE_AGENTS` dictionary:

```
__all__ = [
    # ... existing agents ...
    "MyAwesomeAgent",
    "AVAILABLE_AGENTS",
]
```

Step 3: Run Your Agent

Your agent is now registered and ready to run. Use the class name in lower case as the value for the `--agent` argument.

```
# Run your custom agent on the 'ls20' game
uv run main.py --agent=myawesomeagent --game=ls20
```

You can also run it against all available games:

```
# Run your agent on all games
uv run main.py --agent=myawesomeagent
```

The replay of your agent is available at the end of the run. Make sure to watch your agent at play.

Troubleshooting

Relative Import Errors

If you move an agent file or create a new one outside the `agents/` directory, you may encounter `ImportError` exceptions related to relative imports.

like this:

```
# agents/my_agents/my_file.py

# Correct: Go up one level to the 'agents' package root
from ..agent import Agent
from ..structs import FrameData

# Incorrect: Assumes the file is in the 'agents' root
# from .agent import Agent
```

Agent Not Found Errors

If you see `ValueError: Agent '<your-agent>' not found`, double-check the following:

1. Your agent class is correctly located in the `agents` directory (or a subdirectory).
2. The class name is correctly spelled and matches the name you provided to the `--agent` flag (in lower case).
3. You have saved your changes to your agent file.

Was this page helpful? Yes No Suggest edits

‹ GPT OSS on Kaggle Local vs Online ›